KIA Dealer Management System

Full-Stack Project Documentation

Project Name	KIA Dealer Management System (DMS)
Frontend	React 19, TypeScript, Tailwind CSS
Backend	Spring Boot, Spring Security, JWT
Database	MySQL (via MySQL Shell CLI)
Cache Layer	Redis

ADMIN MANAGER DEALER

Contents

1. Project Overview

The KIA Dealer Management System (DMS) is a full-stack enterprise web application designed to streamline the operations of KIA dealerships across sales, inventory, service, finance, and analytics. The system enforces strict role-based access control (RBAC) for three user roles — Admin, Manager, and Dealer.

1.1. Core Objectives

- Centralise dealership operations in a single, secure platform.
- Enforce role-based access at both route and component levels.
- Demonstrate production-grade technical outcomes: CRUD, pagination, caching, validation, authentication, authorisation, sorting, logging, and exception handling.
- Maintain scalability, reliability, and security at every layer of the stack.

1.2. User Roles & Permissions

Feature	Admin	Manager	Dealer
Dashboard & Analytics	✓	✓	✓
Inventory Management	✓	✓	✓
Sales & Orders	✓	✓	✓
Leads & Test Drives	✓	✓	✓
Service & Workshop	✓	✓	✓
Parts & Suppliers	✓	✓	✓
Finance Module	✓	✓	×
Delete Records	✓	×	×
Admin Panel (Users/Dealers)	✓	×	×

1.3. Expected Technical Outcomes

Query DSL | Scalability | Security (Method-level) | Authentication | Validation (Client/Server) | Authorisation | Performance | Reliability | Sorting | CRUD | Exception Handling | Caching (Internal/External/Browser) | Log Management | Design Pattern (MVC) | Pagination (Client/Server) | Responsiveness | Password Hashing/Encryption

2. System Architecture

2.1. Full-Stack Architecture Overview

The system follows a layered, modular architecture. The React frontend communicates with the Spring Boot backend exclusively via RESTful APIs (secured by JWT).

```
[ React + TypeScript + Tailwind (Browser) ]
            |  HTTPS / REST API
[ Spring Boot Backend (REST Controllers) ]
            |
[ Service Layer (Business Logic + RBAC) ]
            |
[ Repository Layer (JPA + QueryDSL) ]
            |
[ MySQL Database ]     <-->     [ Redis Cache ]
```

2.2. Design Pattern

The backend strictly follows the **MVC (Model-View-Controller)** pattern extended with a Service layer, making it a **Controller – Service – Repository** pattern (Spring Boot standard):

- **Controller** – Handles HTTP requests, validates input, delegates to service.
- **Service** – Contains all business logic, enforces RBAC via `@PreAuthorize`.
- **Repository** – Data access via JPA, with QueryDSL for dynamic query construction.
- **DTO/Mapper** – Separates internal entity models from API request/response objects.

2.3. Security Architecture

- **Authentication:** JWT (JSON Web Token) issued upon login, stored in `localStorage`, sent as `Authorization: Bearer <token>` on every API call.
- **Authorisation:** Method-level security with `@PreAuthorize("hasRole('ADMIN')")` annotations.
- **Password Security:** BCrypt hashing via `BCryptPasswordEncoder`. Passwords require a minimum of 8 characters, one uppercase, one lowercase, one digit, and one special character.
- **Session Management:** Token expires after 30 minutes of inactivity.
- **Frontend Guard:** Protected routes using React Router's `ProtectedRoute` and `RoleRoute` components.

3. Frontend Structure

3.1. Technology Stack

Technology	Purpose
React 19 + TypeScript	UI framework with type safety
Tailwind CSS	Utility-first responsive styling
React Router v6	Client-side routing & protected routes
React Hook Form + Zod	Form management & schema validation
TanStack React Query	Server-state management & API caching
Axios	HTTP client with JWT interceptors
Recharts	Dashboard data visualisation
Zustand	Client-side UI state management
Framer Motion	Modal animations
Lucide Icons	Icon library
Google reCAPTCHA v2	Bot prevention on login
Vite	Build tool and dev server

3.2. Folder Structure

```
kia-dms-frontend/
|-- public/
|   |-- favicon.svg
|   `-- icons.svg
|-- src/
|   |-- assets/
|   |-- components/
|   |   |-- layout/
|   |   |   |-- Sidebar.tsx
|   |   |   |-- Navbar.tsx
|   |   |   `-- Layout.tsx
|   |   `-- ui/
|   |       |-- Button.tsx  |-- Input.tsx  |-- Modal.tsx
|   |       |-- Card.tsx    |-- Badge.tsx  |-- Loader.tsx
|   |       |-- Table/      `-- Charts/
|   |-- context/
|   |   |-- AuthContext.tsx      # Auth + RBAC state
|   |   `-- ThemeContext.tsx     # Dark mode
|   |-- hooks/
|   |   |-- useAuth.ts  |-- useRole.ts  |-- usePagination.ts
|   |-- pages/
|   |   |-- auth/
|   |   |   |-- LoginPage.tsx
|   |   |   `-- SignUpPage.tsx
|   |   |-- dashboard/Dashboard.tsx
|   |   |-- inventory/  |-- sales/  |-- leads/
|   |   |-- service/    |-- parts/  |-- finance/
|   |   |-- analytics/  `-- admin/
|   |-- routes/
|   |   |-- AppRoutes.tsx
```

```
|   |   |-- ProtectedRoute.tsx
|   |   `-- RoleRoute.tsx
|   |-- services/
|   |   |-- api.ts                # Axios + JWT interceptor
|   |   |-- auth.service.ts
|   |   |-- inventory.service.ts
|   |   `-- (one per module)
|   |-- store/
|   |   `-- uiStore.ts            # Zustand store
|   |-- types/                    # TypeScript interfaces
|   |-- utils/
|   |   |-- permissions.ts        # Role permission maps
|   |   |-- validators.ts
|   |   `-- formatters.ts
|   |-- config/
|   |   `-- queryClient.ts
|   |-- App.tsx
|   `-- main.tsx
|-- tailwind.config.js
|-- tsconfig.json
`-- vite.config.ts
```

Listing 1: Frontend Folder Structure

3.3. Frontend Technical Outcomes

Requirement	Implementation
Validation (Client)	Zod schema + React Hook Form
Authentication state	JWT stored in `localStorage`, React Context
Authorisation	`ProtectedRoute`, `RoleRoute`, conditional rendering
Pagination	`usePagination` hook; server-side via `Pageable`
Sorting	`DataTable` with click-to-sort column headers
Caching (Browser)	React Query stale-while-revalidate strategy
Responsiveness	Tailwind breakpoints: sm / md / lg / xl
CRUD	Mutations via React Query + Axios
Security (Login)	Google reCAPTCHA v2 on login form
Performance	React `lazy()` route splitting, React Query cache
Pattern	Functional components, hooks, props typed with TS

3.4. Key Frontend Conventions

- All components are **functional components** (no class components).
- Props are **always typed** using TypeScript interfaces.
- **Destructuring** is used for all props and state (industry standard).
- Lists are always rendered using `map()` with a unique `key` prop.
- Conditional rendering via **ternary operators** and **short-circuit evaluation**.
- **Zod** schemas validate all form inputs before submission.

- **reCAPTCHA v2** on the Login page verifies human users before authentication.
- **Reusable component library** in `src/components/ui/` for buttons, inputs, modals, badges, and tables.

3.5. Module Pages

Module	Page(s)	Features
Dashboard	Dashboard.tsx	KPI cards, Recharts bar/line charts
Inventory	Inventory.tsx, Add/Edit Vehicle	Search, filter, CRUD, table
Sales	Sales.tsx, Orders.tsx, Invoice.tsx	Order management, status tracking
Leads	Leads.tsx, TestDrive.tsx	Lead pipeline, test drive scheduling
Service	Service.tsx, Appointments.tsx	Workshop jobs, status updates
Parts	Parts.tsx, PurchaseOrders.tsx	Stock management, supplier listing
Finance	Finance.tsx, Transactions.tsx	Transactions, reports (restricted)
Analytics	Analytics.tsx, DealerPerformance.tsx	Charts, performance metrics
Admin	Users.tsx, Dealers.tsx, Roles.tsx	User & dealer management
Auth	LoginPage.tsx, SignUpPage.tsx	Role tabs, Zod, reCAPTCHA

4. Backend Structure

4.1. Technology Stack

Technology	Purpose
Spring Boot 3.x	Application framework
Spring Security	Authentication & method-level authorisation
JWT (jjwt)	Stateless token-based authentication
Spring Data JPA + Hibernate	ORM, entity management
QueryDSL	Type-safe dynamic query construction
MySQL 8+ (via MySQL Shell CLI)	Primary relational database
Redis	External distributed cache
Caffeine	Internal in-memory cache
Logback	Structured log management
Spring AOP	Cross-cutting concerns (logging)
BCryptPasswordEncoder	Password hashing
ModelMapper	DTO↔Entity mapping

4.2. Folder Structure

```
kia-dms-backend/
‘-- src/main/java/com/kia/dms/
    |-- KIADealerManagementApplication.java
    |-- config/
    |   |-- AppConfig.java      |-- CorsConfig.java
    |   |-- CacheConfig.java    ‘-- ModelMapperConfig.java
    |-- security/
    |   |-- SecurityConfig.java
    |   |-- JwtAuthenticationFilter.java
    |   |-- JwtTokenProvider.java
    |   |-- CustomUserDetailsService.java
    |   ‘-- PasswordEncoderConfig.java
    |-- exception/
    |   |-- GlobalExceptionHandler.java
    |   |-- ResourceNotFoundException.java
    |   ‘-- ErrorResponse.java
    |-- common/
    |   |-- enums/          (Role, OrderStatus, VehicleStatus)
    |   |-- util/           (DateUtil, JwtUtil, ValidationUtil)
    |   ‘-- response/       (ApiResponse, PaginationResponse)
    |-- audit/
    |   |-- BaseEntity.java      (id, createdAt, updatedAt,
       isDeleted)
    |   ‘-- AuditService.java
    |-- cache/
    |   |-- CacheService.java
    |   ‘-- RedisConfig.java
    |-- modules/
    |   |-- auth/      |-- user/       |-- dealer/
    |   |-- vehicle/   |-- inventory/  |-- sales/
```

```
    |   |-- leads/    |-- service/   |-- parts/
    |   |-- finance/  `-- analytics/
    |
    |   (Each module contains:)
    |   |-- controller/*Controller.java
    |   |-- service/*Service.java + impl/*ServiceImpl.java
    |   |-- repository/*Repository.java
    |   |-- repository/custom/*CustomRepositoryImpl.java
        (QueryDSL)
    |   |-- entity/*Entity.java
    |   |-- dto/request/*Request.java + response/*Response.java
    |   |-- mapper/*Mapper.java
    |   `-- specification/*Specification.java
    |-- scheduler/
    |   |-- InventoryScheduler.java
    |   `-- AnalyticsScheduler.java
    `-- logging/
        `-- LoggingAspect.java  (AOP @Around logging)

`-- src/main/resources/
    |-- application.yml
    |-- application-dev.yml
    |-- application-prod.yml
    `-- logback-spring.xml
```

Listing 2: Backend Folder Structure (Spring Boot)

4.3. Backend Technical Outcomes

Requirement	Implementation
Query DSL	`BooleanBuilder` in `*CustomRepositoryImpl`
Scalability	Modular domain packages, stateless APIs
Security (Method-level)	`@PreAuthorize("hasRole('ADMIN')")`
Authentication	JWT via `JwtAuthenticationFilter`
Validation (Server)	Hibernate Validator (`@NotNull`, `@Size`) on DTOs
Authorisation	RBAC enforced in service layer
Performance	Lazy loading, Redis cache, DB indexing
Reliability	`@Transactional`, retry logic, exception handler
Sorting	`Sort.by()` via `Pageable`
CRUD	Full in every module
Exception Handling	`@RestControllerAdvice GlobalExceptionHandler`
Caching (Internal)	Caffeine (`@Cacheable("vehicles")`)
Caching (External)	Redis distributed cache
Log Management	Logback + AOP `@Around` logging
Pattern	MVC + Service + Repository layers
Pagination	`PageRequest.of(page, size, sort)`
Password Encryption	BCrypt hashing

5. Database Schema

The database is **MySQL 8+**, accessed via the **MySQL Shell CLI**. It follows Third Normal Form (3NF), includes audit timestamps, and uses soft-delete where appropriate. All foreign key constraints are explicitly defined.

```
CREATE DATABASE kia_dms;
USE kia_dms;
```

5.1. Table: roles

Stores the three system roles. Referenced by every user record to enforce RBAC.

```
CREATE TABLE roles (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(50)  UNIQUE NOT NULL,  -- ADMIN | MANAGER
        | DEALER
    description VARCHAR(255),
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.2. Table: dealers

Represents a KIA dealership outlet. Users and operational data (orders, inventory, leads, transactions) are all scoped to a dealer.

```
CREATE TABLE dealers (
    id             BIGINT PRIMARY KEY AUTO_INCREMENT,
    name           VARCHAR(150) NOT NULL,
    location       VARCHAR(255),
    contact_number VARCHAR(20),
    email          VARCHAR(100),
    status         VARCHAR(50),   -- ACTIVE | INACTIVE
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
        CURRENT_TIMESTAMP
);
```

5.3. Table: users

Holds all system users. The `role_id` FK drives RBAC; `dealer_id` scopes dealer-role users to their own dealership.

```
CREATE TABLE users (
    id         BIGINT PRIMARY KEY AUTO_INCREMENT,
    name       VARCHAR(100) NOT NULL,
    email      VARCHAR(100) UNIQUE NOT NULL,
    password   VARCHAR(255) NOT NULL,  -- BCrypt hashed
    role_id    BIGINT NOT NULL,
    dealer_id  BIGINT NULL,
    is_active  BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
        CURRENT_TIMESTAMP,
    FOREIGN KEY (role_id)   REFERENCES roles(id),
    FOREIGN KEY (dealer_id) REFERENCES dealers(id)
);
```

5.4. Table: vehicles

Master catalogue of all KIA vehicle models. Used as a reference in inventory, orders, and service.

```
CREATE TABLE vehicles (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100),
    model       VARCHAR(100),
    price       DECIMAL(10,2),
    category    VARCHAR(50),     -- SUV | SEDAN | HATCHBACK
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.5. Table: inventory

Tracks stock levels for each vehicle at each dealer. Updated automatically when orders are placed.

```
CREATE TABLE inventory (
    id           BIGINT PRIMARY KEY AUTO_INCREMENT,
    vehicle_id   BIGINT NOT NULL,
    dealer_id    BIGINT NOT NULL,
    quantity     INT,
    status       VARCHAR(50),   -- AVAILABLE | LOW_STOCK |
       OUT_OF_STOCK
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
       CURRENT_TIMESTAMP,
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id),
    FOREIGN KEY (dealer_id)  REFERENCES dealers(id)
);
```

5.6. Table: orders

Sales orders placed by dealers for vehicles. Linked to dealer and vehicle; drives revenue analytics.

```
CREATE TABLE orders (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    dealer_id   BIGINT NOT NULL,
    vehicle_id  BIGINT NOT NULL,
    quantity    INT,
    total_price DECIMAL(12,2),
    status      VARCHAR(50),   -- PENDING | COMPLETED | CANCELLED
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dealer_id)  REFERENCES dealers(id),
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id)
);
```

5.7. Table: leads

Prospective customer records. Tracks the vehicle interest and conversion status for each lead at a dealership.

```
CREATE TABLE leads (
    id               BIGINT PRIMARY KEY AUTO_INCREMENT,
    customer_name    VARCHAR(100),
    contact          VARCHAR(50),
    vehicle_interest VARCHAR(100),
```

```
    status            VARCHAR(50),   -- NEW | FOLLOWUP | CONVERTED
    dealer_id         BIGINT NOT NULL,
    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dealer_id) REFERENCES dealers(id)
);
```

5.8. Table: test_drives

Schedules test drives for leads. A single lead can have multiple test drive appointments.

```
CREATE TABLE test_drives (
    id             BIGINT PRIMARY KEY AUTO_INCREMENT,
    lead_id        BIGINT NOT NULL,
    schedule_date  DATETIME,
    status         VARCHAR(50),   -- SCHEDULED | COMPLETED |
       CANCELLED
    FOREIGN KEY (lead_id) REFERENCES leads(id)
);
```

5.9. Table: service_orders

Records vehicle service and repair jobs at a dealership workshop.

```
CREATE TABLE service_orders (
    id           BIGINT PRIMARY KEY AUTO_INCREMENT,
    dealer_id    BIGINT NOT NULL,
    vehicle_id   BIGINT NOT NULL,
    description TEXT,
    status       VARCHAR(50),   -- IN_PROGRESS | COMPLETED | PENDING
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dealer_id)  REFERENCES dealers(id),
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id)
);
```

5.10. Table: parts

Spare parts and accessories catalogue with stock levels and supplier information.

```
CREATE TABLE parts (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100),
    price       DECIMAL(10,2),
    stock       INT,
    supplier    VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.11. Table: purchase_orders

Represents restock orders raised by dealers to procure spare parts from suppliers.

```
CREATE TABLE purchase_orders (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    part_id     BIGINT NOT NULL,
    quantity    INT,
    total_cost DECIMAL(12,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (part_id) REFERENCES parts(id)
);
```

5.12. Table: transactions

Financial transactions (credit/debit) associated with each dealer. Accessible only by Admin and Manager.

```
CREATE TABLE transactions (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    dealer_id   BIGINT NOT NULL,
    amount      DECIMAL(12,2),
    type        VARCHAR(50),    -- CREDIT | DEBIT
    description VARCHAR(255),
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dealer_id) REFERENCES dealers(id)
);
```

5.13. Table: dealer_performance

Aggregated analytics data per dealer (computed by the scheduler). Powers the Analytics module.

```
CREATE TABLE dealer_performance (
    id              BIGINT PRIMARY KEY AUTO_INCREMENT,
    dealer_id       BIGINT NOT NULL,
    sales_count     INT,
    revenue         DECIMAL(12,2),
    conversion_rate DECIMAL(5,2),
    score           INT,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (dealer_id) REFERENCES dealers(id)
);
```

5.14. Indexes for Performance

```
CREATE INDEX idx_vehicle_name   ON vehicles(name);
CREATE INDEX idx_orders_dealer  ON orders(dealer_id);
CREATE INDEX idx_inventory_vid  ON inventory(vehicle_id);
CREATE INDEX idx_leads_dealer   ON leads(dealer_id);
```

5.15. Entity Relationship Summary

Parent Table	Child Table(s)	Relationship
roles	users	One-to-Many
dealers	users, inventory, orders	One-to-Many
dealers	leads, transactions, service_orders	One-to-Many
vehicles	inventory, orders, service_orders	One-to-Many
leads	test_drives	One-to-Many
parts	purchase_orders	One-to-Many

6. Application Flow

6.1. Authentication Flow

1. User navigates to `/login` and selects role tab (Dealer / Manager / Admin).
2. Fills credentials; Zod validates on client side. reCAPTCHA verifies human.
3. Axios sends `POST /api/auth/login` with `{email, password}`.
4. Backend validates credentials, verifies BCrypt hash, issues JWT.
5. Frontend stores JWT in `localStorage`; React context updates user state.
6. React Router's `ProtectedRoute` grants access; sidebar renders role-appropriate menu.

6.2. CRUD Flow (Example: Inventory)

1. User navigates to Inventory page. React Query fires `GET /api/inventory`.
2. Backend `InventoryController` delegates to `InventoryServiceImpl`.
3. Service calls QueryDSL-backed `InventoryCustomRepository` with filters.
4. Paginated, sorted results returned as `PaginationResponse<InventoryResponse>`.
5. Frontend renders data in `DataTable` with sort headers and pagination controls.
6. Admin/Manager clicks "Add" – form modal opens, Zod validates, mutation fires `POST`.
7. On success, React Query invalidates cache; table auto-refreshes. Toast notification shown.

7. Caching Strategy

Cache Type	Tool	Usage
Internal (JVM)	Caffeine	Vehicle catalogue, roles list
External	Redis	Session state, frequently accessed dealer data
Browser	React Query (stale-while-revalidate)	API response caching on client
HTTP	Cache-Control headers	Static assets, logo, icons

8. Test Credentials

Role	User ID	Email	Password
Admin	`admin`	admin@kia-dms.com	`Admin@123`
Manager	`manager`	manager@kia-dms.com	`Manager@123`
Dealer	`dealer`	dealer@kia-dms.com	`Dealer@123`

All passwords meet requirements: minimum 8 characters, at least one uppercase letter, one lowercase letter, one digit, and one special character.

9. Technical Outcomes Summary

Outcome	Frontend	Backend
Query DSL	React Query dynamic params	QueryDSL `BooleanBuilder`
Scalability	Modular pages, lazy loading	Stateless APIs, modular packages
Security	JWT context, RoleRoute guards	Spring Security, `@PreAuthorize`
Authentication	JWT stored in localStorage	JWT filter, token validation
Validation	Zod + React Hook Form	Hibernate Validator on DTOs
Authorisation	Conditional rendering + RBAC	Method-level `@PreAuthorize`
Performance	React Query cache, code splitting	Redis, Caffeine, lazy JPA fetch
Reliability	Error boundaries, toast alerts	`@Transactional`, global handler
Sorting	DataTable column click sort	`Sort.by()` via `Pageable`
CRUD	Mutations via React Query	Full in all module services
Exception Handling	Axios error interceptor	`@RestControllerAdvice`
Caching	React Query stale-while-revalidate	Redis + Caffeine + HTTP headers
Log Management	Browser console	Logback + AOP `@Around`
Pattern	Functional components + hooks	MVC + Service + Repository
Pagination	`usePagination` hook	`PageRequest` + `PaginationResponse`
Responsiveness	Tailwind sm/md/lg/xl	N/A
Password Encryption	Client-side validation only	BCrypt (`BCryptPasswordEncoder`)